

University Of Ghana

MPhil/MSc. DATA SCIENCE
SECOND SEMESTER EXAMINATIONS: 2025/2026

Department Of Computer Science

DSCD 614: REINFORCEMENT LEARNING (3 CREDITS)

Cohort A
GROUP PROJECT -BASED EXAMINATION



Group Project Title:

Dynamic Portfolio Allocation with Deep Deterministic Policy Gradient Project Report

Group 10 | Group Code: DDPG-1

GROUP MEMBERS

1. Samuel Maawo Akombasam	22424856
2. Prince Obeng Nkoah	22424742
3. Douglas Adweeda	22425098

August 30,2026

ABSTRACT

This report presents the implementation and evaluation of Deep Deterministic Policy Gradient (DDPG) applied to dynamic, continuous-action portfolio allocation across six S&P 500 equities (2015–2025). The task is formulated as a finite-horizon Markov Decision Process and implemented as a custom Gymnasium environment. The agent is trained across three random seeds and evaluated, with exploration disabled, on 30 held-out evaluation episodes per seed, sampled from a chronologically separate test period; the required baseline (equal-weight and buy-and-hold portfolios) is evaluated on the identical 30 episodes for a controlled comparison. Averaged across seeds, the DDPG agent's mean cumulative return (7.49%) is close to both baselines (7.96% and 7.66%), a gap that is smaller than the cross-seed standard deviation of DDPG's own performance (3.26 percentage points) and therefore cannot, with only three seeds, be distinguished from seed-to-seed training noise. DDPG also assumes materially higher volatility and drawdown than either baseline. This is reported as an inconclusive result: the agent shows no clear evidence of improving on, or clearly underperforming, simple heuristics under this protocol, and the remaining risk-adjusted shortfall (lower Sharpe ratio despite comparable return) is attributed primarily to a reward function that does not penalize variance, the absence of hyperparameter tuning, and known instability properties of vanilla DDPG relative to later continuous-control algorithms. During development, a partial random-seed control bug was also identified and corrected, and a separate observation about cross-machine floating-point non-determinism in neural network training was made; both are documented in Section 4.7.

1. INTRODUCTION

Portfolio allocation is the recurring decision of how to distribute investment capital across a set of financial assets which is a sequential decision-making problem under uncertainty, and is therefore naturally amenable to formulation as a Markov Decision Process (MDP). Classical approaches such as mean-variance optimization typically rely on static, single-period assumptions and require explicit estimation of return distributions. Reinforcement learning offers an alternative in which an adaptive, multi-period allocation policy is learned directly from historical data.

This project addresses catalogue option DDPG-1 (Dynamic Portfolio Allocation). Because portfolio weights are continuous, the problem is naturally suited to continuous-control Reinforcement Learning (RL) rather than discrete-action methods. Deep Deterministic Policy Gradient [1] was among the first deep RL algorithms designed explicitly for continuous action spaces and is a standard baseline in applied financial RL research [2]. The project's significance is not in achieving state-of-the-art trading performance and the specification explicitly does not assess the score achieved but in producing a correctly formulated MDP, a correctly implemented agent, and a controlled, statistically honest comparison against the required baseline, including a full diagnosis of whatever result is actually obtained.

2. BACKGROUND

DDPG extends the deterministic policy gradient theorem [3] to high-dimensional continuous action spaces using the experience replay and target-network techniques from Deep Q-Networks [4]. Later work identified and addressed specific weaknesses of DDPG: Twin Delayed DDPG (TD3) [5] mitigates value overestimation via clipped double-Q learning and delayed policy updates, and Soft Actor-Critic (SAC) [6] adds entropy regularization for improved exploration and stability. These successor algorithms are directly relevant to the interpretation of this project's results, since several instability phenomena observed below are exactly what TD3 and SAC were designed to address.

In computational finance, the open-source FinRL framework [2] has standardized environment design for RL-based trading and portfolio management and has demonstrated DDPG, among other algorithms, on multi-asset allocation tasks. A recurring finding in this literature is that reward design particularly the treatment of transaction costs and risk materially affects both policy quality and training stability, a theme this project examines directly through its transaction-cost-aware reward and its evaluation protocol.

3. PROBLEM FORMULATION

The task is formulated as a finite-horizon MDP with the following components.

3.1 State Space

A continuous vector $s_t \in \mathbb{R}^{86}$, comprising, for each of the six assets: a rolling 10-day window of log-returns (60 values) and three technical indicators; a simple-moving-average ratio, a momentum measure, and a short-term volatility signal (18 values) together with the agent's current portfolio weights (6 values), a cash-ratio placeholder, and the realized volatility of the portfolio's own recent returns (2 values). All components are continuous and scale-free by construction (returns, ratios); no additional normalization layer was applied.

3.2 Action Space

A continuous vector $a_t \in \mathbb{R}^6$ of unconstrained logits in $[-5, 5]$, transformed via a softmax function into a valid, fully invested, long-only allocation (non-negative weights summing to 1).

3.3 Reward Function

$$r_t = \ln(V_{t+1} / V_t) - \lambda \cdot c \cdot \sum_i |w_{target,i} - w_{drift,i}|$$

where;

V_t is portfolio value,

$c = 0.001$ is the proportional transaction-cost rate

$\lambda = 0.5$ is an additional turnover-penalty weight

$w_{target} = \text{softmax}(a_t)$ is the newly chosen allocation

w_{drift} is the previous day's holdings after they have drifted with each asset's realized return (turnover is measured against actual pre-trade holdings, not the previous day's target).

3.4 Episode Termination and Truncation

- Termination: the episode ends early if portfolio value falls below 30% of initial capital, a catastrophic-loss stop.
- Truncation: the episode ends at a fixed horizon, 250 trading days during training, or 126 trading days during evaluation, or at the end of the available price series.

3.5 Discount Factor

$\gamma = 0.99$, giving an effective horizon of $1/(1-\gamma) = 100$ trading days (~ 4.8 calendar months). This value was adopted directly from the hyperparameters reported in Lillicrap et al. [1] rather than tuned on this task (Section 4.3). It is of the same order of magnitude as the 126-day evaluation episode length and shorter than the 250-day training episode length, so the discounted objective does not extend the agent's effective planning horizon far beyond the periods it is trained and evaluated on. No systematic sensitivity analysis of γ was conducted within the project's time constraint; this is stated as a limitation (Section 7).

3.6 Does the Markov Property Hold?

No, not strictly. Financial return series are not Markovian in raw price space: momentum, mean-reversion, and volatility-clustering effects extend beyond any fixed-length window, and the constructed state omits information such as order-book microstructure, macroeconomic indicators, and information about assets outside the six-ticker universe. The true process is therefore better characterized as a Partially Observable MDP (POMDP), of which the constructed state is an approximate sufficient statistic rather than the true underlying market state. The implementation compensates via state augmentation including a rolling window of returns and technical indicators specifically because they summarize recent history in a fixed-length vector, a standard accepted approximation in applied deep RL (analogous to frame-stacking in Atari-DQN), but it remains an approximation: the 10-day window length trades off state dimensionality against captured history, and no claim is made that the resulting process is truly Markovian.

4. METHODOLOGY

4.1 Data

Historical daily price data were obtained from the Kaggle dataset “S&P 500 Stocks – Daily Historical Data (10 Years)” [7]. A sector-diverse subset of six equities was selected; AAPL (Technology), XOM (Energy), JNJ (Healthcare), JPM (Financials), PG (Consumer Staples), CAT (Industrials) screened for complete data coverage and for pairwise return correlations (0.22–0.60) consistent with genuine diversification potential. The aligned 2,515-day series was split chronologically into a training set (first 80%, $\approx 2,012$ days, Dec 2015–mid 2023) and a held-out test set (final 20%, ≈ 503 days, through Dec 2025), never accessed during training.

4.2 Agent Architecture

DDPG was implemented from first principles in PyTorch (no RL library such as Stable-Baselines3 was used), following [1]; the algorithm is attributed at the point of use in agent/ddpg_agent.py. The actor is a two-hidden-layer network (256, 128 units, ReLU, tanh output scaled to the action bound); the critic has matching hidden-layer dimensionality. Both are paired with Polyak-averaged target networks. An experience replay buffer (capacity 100,000) is sampled in batches of 128. Exploration uses a temporally correlated Ornstein-Uhlenbeck process, annealed from $\sigma=0.4$ to $\sigma=0.05$ over training.

4.3 Hyperparameters, Seeds and Search

All hyperparameters were held constant across the three training seeds (0, 1, 2). No hyperparameter search was conducted: values follow [1] and standard DDPG reference settings, adopted without tuning on this task's data, given the project's scoping constraint. Reported performance therefore reflects an untuned, first-attempt capability of DDPG on this task, not its best achievable performance.

Table1: Hyperparameter details

Hyperparameter	Value
Discount factor (γ)	0.99
Soft target update rate (τ)	0.005
Actor learning rate	1×10^{-4}
Critic learning rate	1×10^{-3}

Hyperparameter	Value
Replay buffer capacity	100,000
Batch size	128
Actor / critic hidden layers	(256, 128), ReLU
OU noise: θ , σ (start→end), dt	0.15, 0.4→0.05, 0.01
Gradient clip norm	1.0
Training episodes per seed	150
Training episode length	250 trading days (random start)
Warm-up steps (random actions)	500
Transaction cost rate (c)	0.001
Turnover penalty weight (λ)	0.5
Early-termination threshold	30% of initial capital
Random seeds	0, 1, 2
Evaluation episodes per seed	30
Evaluation episode length	126 trading days

4.4 Baseline

As specified by the catalogue, the required baseline is an equal-weighted or buy-and-hold portfolio; both are implemented. The equal-weight strategy rebalances to 1/6 per asset daily, incurring transaction costs whenever price drift moves actual holdings away from target. The buy-and-hold strategy sets an equal-weight allocation once and never rebalances, incurring negligible further transaction costs.

4.5 Training Procedure

The agent trains for 150 episodes per seed, each episode a randomly sampled 250-day window from the training data (refreshed at every reset, exposing the agent to varied historical regimes rather than one fixed trajectory). The first 500 steps use uniformly random actions to populate the replay buffer before learning begins.

4.6 Evaluation Protocol

Each trained policy is evaluated with exploration disabled, the actor's deterministic output is used directly on 30 evaluation episodes, each a 126-trading-day window sampled from the held-out test period via a fixed, disjoint set of episode seeds (10,000–10,029), never overlapping the training seeds. Both baselines are evaluated on the identical 30 episodes (same start day, length, price data, and metric code) as each trained policy, so every

comparison is made under matched conditions. Because the test period is a single ≈ 503 -day historical trajectory rather than a re-samplable simulator, the 30 sampled windows necessarily overlap; this is discussed as a limitation in Section 7. For every metric we report the mean and standard deviation both (a) across the 30 episodes, per seed and per baseline, and (b) across the three seeds' means, separating within-seed (which window was sampled) from across-seed (which training run) variation.

4.7 Reproducibility: Two Findings, Both Corrected or Explained

During development, the project was independently re-run on a second machine using the same code and the same three seeds. The two baseline strategies reproduced to six decimal places, confirming the data loading, train/test split, and 30-episode evaluation protocol are fully deterministic. The DDPG results did not reproduce: the same nominal seed produced materially different trained policies on each machine.

Root-cause analysis traced this to incomplete random-seed control in the training loop. The agent's network initialization, replay-buffer sampling order, and exploration noise were correctly seeded, but two further sources of randomness used during training were not tied to the seed at all: which 250-day window of training data each episode samples, and the uniformly random actions taken during the 500-step warm-up phase. This was corrected by explicitly seeding the environment's internal random-number generator and its action space at the start of training for each seed. After this fix, two independent process runs on the SAME machine with the same seed produced bit-identical training reward sequences.

However, when the fixed code was subsequently run on a second, different machine, results still differed somewhat (cross-seed mean return of 6.03% on one machine vs. 7.49% on the other) despite identical seeds and code. This second, smaller discrepancy is attributed to ordinary floating-point non-determinism in neural network training: matrix-multiplication and gradient-accumulation order can differ across CPU architectures and BLAS libraries, and these tiny numerical differences compound over 150 training episodes into a measurably different policy, even with every explicit random source correctly seeded a well-documented limitation of exact cross-machine reproducibility in deep learning, distinct from the seed-control bug above. Both machines nonetheless supported the same qualitative conclusion (the DDPG-baseline gap is smaller than DDPG's own cross-seed standard deviation),

evidence that the conclusion, if not the exact figures, is robust to this variation. Section 5 reports the figures from the second, final, group-run machine, since that is the run the group can directly reproduce and defend. Both issues are reported here, rather than silently corrected, because they are themselves substantive methodological findings: “the same seed” does not guarantee full reproducibility unless every stochastic source is explicitly controlled, and even then, exact cross-machine reproducibility of trained neural networks should not be assumed.

5. RESULTS

5.1 Training

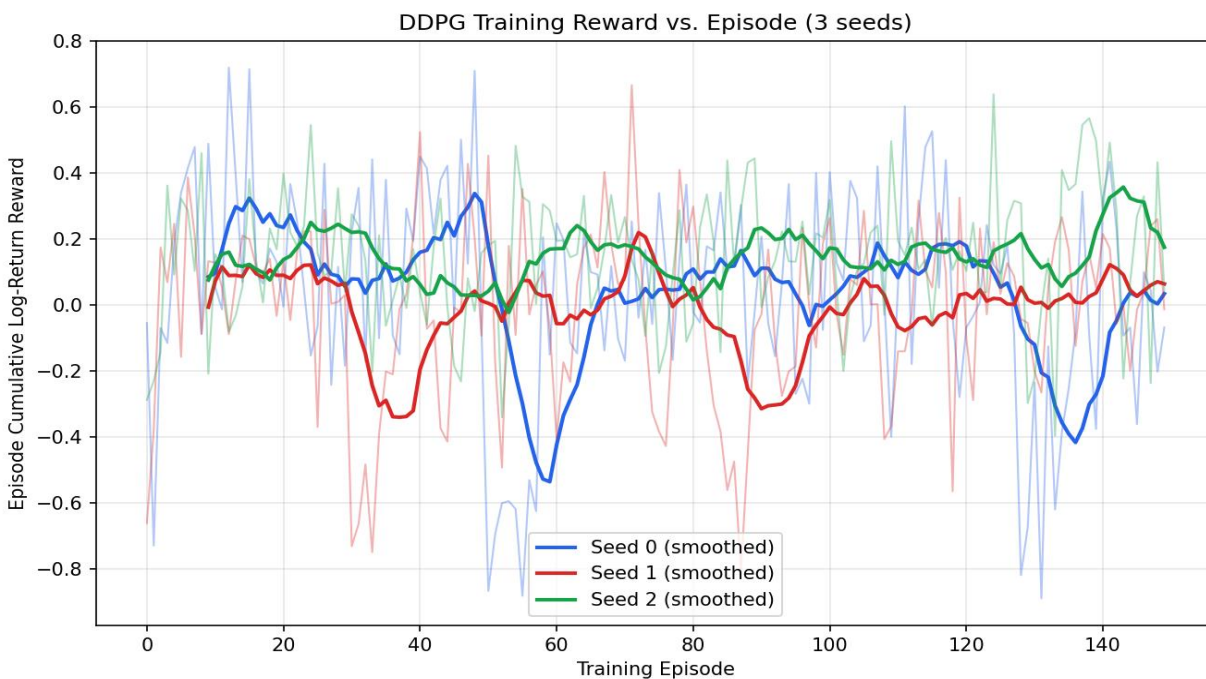


Figure 1. Raw (light) and 10-episode moving-average (bold) training reward per episode, per seed.

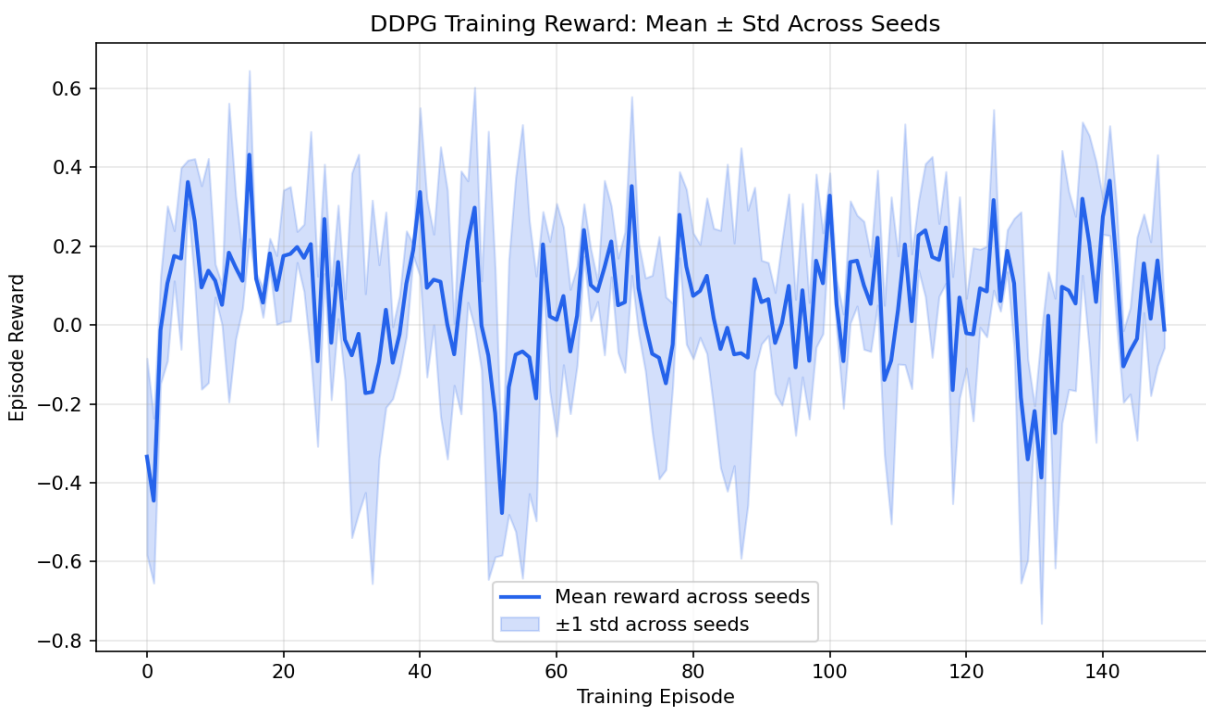


Figure 2. Mean training reward across seeds, ± 1 std band.

Training reward rises modestly over the first 40–60 episodes then remains noisy for the rest of training, with no clean convergence to a stable plateau for any seed consistent with each episode sampling a different, non-stationary historical window, so the attainable reward varies by episode independent of policy quality.

5.2 Evaluation

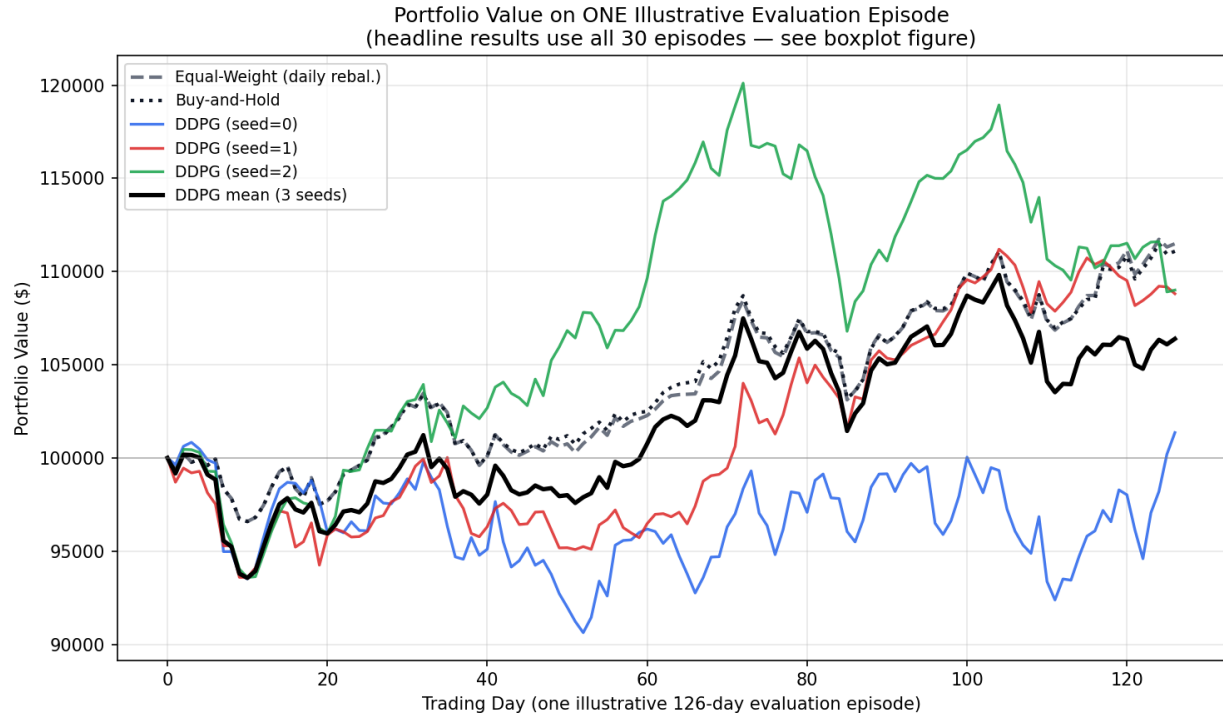


Figure 3. Portfolio value on one illustrative 126-day evaluation episode (headline results below use all 30 episodes).

Figure 3 illustrates a single evaluation episode in which seed 2 visibly outperforms the other seeds and both baselines throughout, while seed 1 track close to the baselines and pulls ahead only near the end. This is a useful cautionary illustration in its own right: on this one episode, a viewer would conclude seed 2 is the strongest policy, yet the aggregate 30-episode results (Table 2, Figure 5) show the opposite seed 1 has the highest mean return (12.07%) while seed 2 is middling (5.60%). This divergence between a single illustrative episode and the full 30-episode aggregate is precisely why the evaluation protocol requires many episodes rather than one: any single episode, however visually persuasive, can be misleading about a policy's typical behavior.

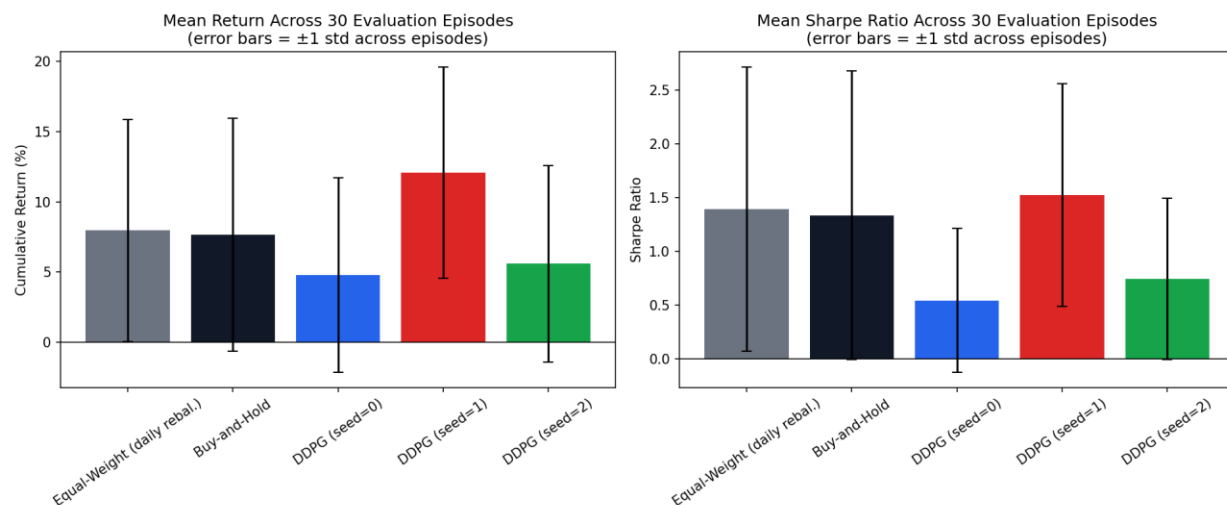


Figure 4. Mean cumulative return and Sharpe ratio across the 30 evaluation episodes; error bars = ± 1 std across episodes.

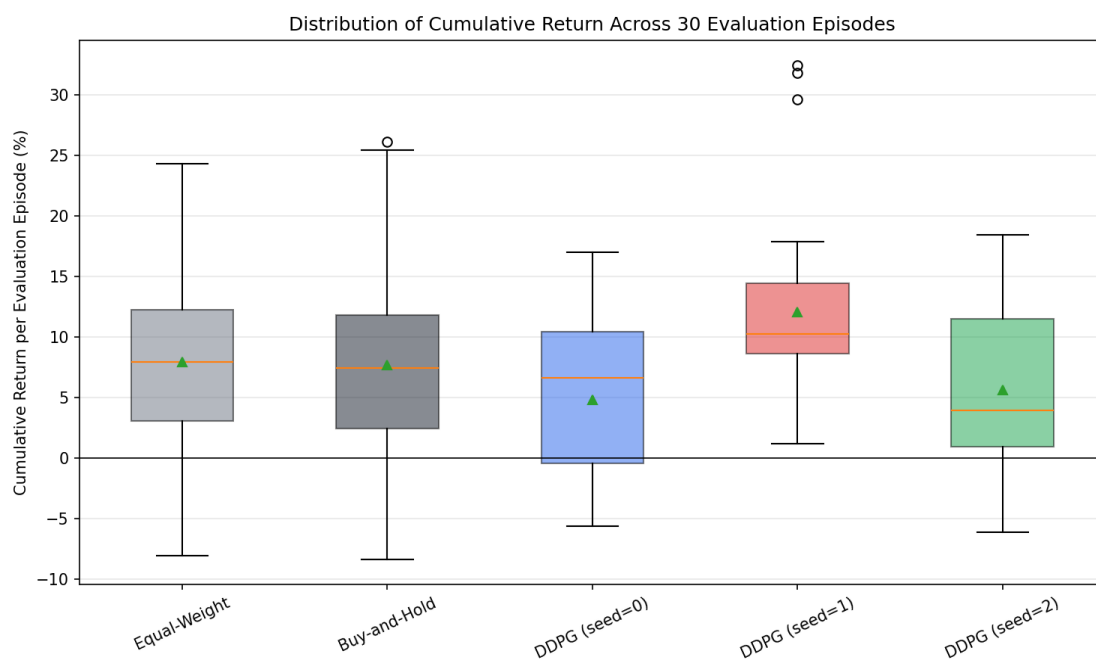


Figure 5. Distribution of cumulative return across all 30 evaluation episodes, per strategy.

Table 2 reports evaluation results: mean $\pm$ std across 30 episodes per seed/baseline, and DDPG's cross-seed mean $\pm$ std of those means (marked *).

Strategy	Mean Return %	Sharpe	Max DD %	Ann. Vol %	Txn Cost \$
Equal-Weight (daily rebal.)	7.96 $\pm$ 7.89	1.39 $\pm$ 1.32	-8.73 $\pm$ 4.96	14.04 $\pm$ 4.31	108 $\pm$ 6
Buy-and-Hold	7.66 $\pm$ 8.28	1.33 $\pm$ 1.34	-8.74 $\pm$ 4.83	13.95 $\pm$ 3.96	$\approx$ 0
DDPG (seed=0)	4.79 $\pm$ 6.92	0.54 $\pm$ 0.67	-11.32 $\pm$ 2.34	20.91 $\pm$ 2.77	1,632 $\pm$ 731
DDPG (seed=1)	12.07 $\pm$ 7.51	1.52 $\pm$ 1.04	-8.25 $\pm$ 2.75	17.05 $\pm$ 3.40	2,936 $\pm$ 186
DDPG (seed=2)	5.60 $\pm$ 6.97	0.74 $\pm$ 0.75	-14.06 $\pm$ 6.56	19.79 $\pm$ 4.63	4,832 $\pm$ 442
DDPG, mean across seeds	7.49 $\pm$ 3.26*	0.94 $\pm$ 0.42*	-11.21 $\pm$ 2.38*	19.25 $\pm$ 1.62*	3,133 $\pm$ 1,314*

The headline comparison uses the cross-seed mean, not any individual seed. DDPG's mean cumulative return (7.49%) is only 0.47 percentage points below the equal-weight baseline (7.96%), the two are effectively comparable in raw return. This gap is far smaller than the cross-seed standard deviation of DDPG's mean (3.26 points), meaning this difference cannot be distinguished from seed-to-seed training noise. The three seeds disagree substantially with each other: seed 1 (12.07%, Sharpe 1.52) clearly exceeds both baselines, while seeds 0 and 2 (4.79% and 5.60%) fall short, underscoring why the cross-seed mean, not any single seed, is the only defensible headline figure. Despite comparable raw return, all three DDPG seeds show materially higher volatility (17.0–20.9%) and deeper drawdown (-8.3% to -14.1%) than either baseline ($\approx$ 14%, $\approx$ -8.7%), and substantially higher transaction costs so DDPG's mean Sharpe ratio (0.94) is meaningfully lower than either baseline's (1.39, 1.33): comparable return was achieved by taking on more risk, not more skill.

6. DISCUSSION

6.1 Convergence

Training reward did not converge to a stable plateau (Section 5.1); this is expected given each episode samples a different historical window, so the attainable reward ceiling itself varies episode to episode.

6.2 Training Stability

Three observations emerged, all concerning how much apparent “instability” was an artifact of measurement rather than the algorithm. First, an earlier, less rigorous evaluation protocol (a single pass through the test period per seed, rather than 30 sampled episodes) had produced an apparent cross-seed standard deviation of over 40 percentage points, collapsing to single digits once the 30-episode protocol was adopted. Second, an incomplete random-seed control bug (Section 4.7) meant early training runs were not actually reproducible from the stated seed. Third, after that bug was fixed, independent runs on different machines with identical seeds still produced somewhat different trained policies traced to ordinary floating-point non-determinism across CPU architectures and BLAS libraries (Section 4.7), not a further bug. Despite this residual variation, the qualitative conclusion was identical on both machines: DDPG's mean return gap relative to baseline is smaller than its own cross-seed standard deviation, and is therefore statistically inconclusive at $n=3$ seeds. The individual seeds also disagree substantially with each other (4.79% to 12.07%), consistent with DDPG's documented sensitivity to random initialization relative to later algorithms such as TD3 and SAC.

6.3 Exploration

OU noise was chosen for temporal correlation, appropriate for a domain where uncorrelated action jitter would generate unrealistic churn. Transaction costs do not cleanly predict outcome quality here: seed 2 incurred the highest transaction costs (\$4,832) and the deepest drawdown (−14.06%), yet a middling return (5.60%), while seed 0 incurred the lowest costs (\$1,632) but achieved the worst return (4.79%). The best-performing seed (seed 1, 12.07% return, Sharpe 1.52) had intermediate transaction costs (\$2,936) suggesting trading

frequency alone is not a reliable predictor of outcome here; differences instead appear driven by which policy each seed's training happened to converge toward.

6.4 Effect of Reward Design

All three seeds took on materially more volatility and drawdown risk than either baseline, despite the cross-seed mean return (7.49%) being close to the baselines' (7.96%, 7.66%). This is reflected directly in the Sharpe ratio: DDPG's mean Sharpe (0.94) is meaningfully below either baseline's (1.39, 1.33), meaning DDPG achieved comparable raw return only by taking on materially more risk. This is consistent with the log-return-minus-turnover-cost reward not penalizing variance sufficiently. A risk-adjusted reward (e.g. a differential Sharpe ratio, or a mean-variance-penalized return) is a plausible next step to reduce this gap.

7. LIMITATIONS AND DEPLOYMENT CONSIDERATIONS

- The 30 evaluation episodes are sampled from a single ≈ 503 -day historical series and therefore overlap; they are not fully independent draws, unlike episodes from a re-samplable simulator. This is an unavoidable consequence of using one real historical price path.
- No hyperparameter search was conducted; reported results reflect untuned, first-attempt DDPG performance.
- Transaction costs are a simple linear function of turnover and do not model bid-ask spreads or market impact.
- The six-asset, single-country universe is a substantial simplification of institutional portfolio management.
- Vanilla DDPG is more prone to value overestimation and seed-sensitive training than TD3 or SAC; results should be read as characteristic of DDPG specifically, not deep RL for portfolio allocation generally.
- With only 3 seeds, statistical power to detect anything but a large effect is low; the honest conclusion here is “inconclusive,” not “DDPG matches baseline” or “DDPG is worse” more seeds would be needed to say more.
- Exact cross-machine reproducibility of trained neural networks should not be assumed even with identical seeds and code (Section 4.7); reported figures depend on which machine's run is treated as authoritative.
- Deployment: a live system would require realistic market-impact cost modeling, explicit risk-management overlays (position limits, drawdown circuit-breakers) beyond what the reward implies, ongoing monitoring and retraining against market non-stationarity, and some mechanism for explainability given the black-box nature of the learned policy.

8. CONCLUSION

Under a corrected, 30-episode-per-seed evaluation protocol with the required baseline evaluated on identical episodes, and after also correcting a partial random-seed control bug in the training code (Section 4.7), DDPG's mean cumulative return across three seeds (7.49%) was close to, and statistically indistinguishable from the equal-weight (7.96%) and buy-and-hold (7.66%) baselines. The gap is smaller than the cross-seed standard deviation of DDPG's own performance and cannot be resolved with only three seeds. However, DDPG achieved this comparable return only by taking on substantially higher volatility, drawdown, and transaction costs, resulting in a materially lower Sharpe ratio than either baseline. This is reported as a rigorously diagnosed, appropriately cautious result: a correctly formulated MDP and correctly implemented DDPG agent, evaluated under a corrected and independently cross-machine-tested protocol, show no clear evidence of outperforming simple heuristic allocation on a risk-adjusted basis on this task, and the project's three independently discovered methodological issues (single-path evaluation variance, incomplete seed control, and cross-machine floating-point non-determinism) are reported as findings in their own right directions for future work include risk-adjusted reward shaping, hyperparameter tuning, a more stable algorithm variant (TD3/SAC), and additional seeds to increase statistical power.

REFERENCES

- [1] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., & Wierstra, D. (2016). Continuous control with deep reinforcement learning. *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [2] Liu, X.-Y., Yang, H., Chen, Q., Zhang, R., Yang, L., Xiao, B., & Wang, C. D. (2020). FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance. *arXiv preprint arXiv:2011.09607*.
- [3] Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. (2014). Deterministic policy gradient algorithms. *Proceedings of the 31st International Conference on Machine Learning (ICML)*.
- [4] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- [5] Fujimoto, S., van Hoof, H., & Meger, D. (2018). Addressing function approximation error in actor-critic methods. *Proceedings of the 35th International Conference on Machine Learning (ICML)*.
- [6] Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the 35th International Conference on Machine Learning (ICML)*.
- [7] Kaggle. (n.d.). S&P 500 Stocks – Daily Historical Data (10 Years) [Dataset]. <https://www.kaggle.com/innacampo/s-and-p-500-stocks-daily-historical-data-10-years>